Computer Vision Engineer: Mask Refinement Pipeline

Overview

In vehicle damage inspection, our models often output noisy segmentation masks. These raw predictions frequently contain artifacts: internal holes (e.g., perhaps due to reflections on a car door), "random" noise (false positives outside the car), and jagged edges.

Your goal is to build a clean, modular Python function to refine these masks, making them suitable for downstream business logic.

The Challenge

Design and implement a Python function that takes a batch of raw binary masks (NumPy arrays) and applies refinement steps to improve their quality.

Core Requirements

1. **Refinement Logic**: Implement at least one or two logical steps. You should consider common geometric issues like:
    - Closing internal holes within a single part.
    - Filtering out small, disconnected polygons based on area.
    - Any other issue that you can think of!
2. **Modularity**: The system should allow for easy "plug-and-play" of different refinement steps. We don't want you to implement everything but we do want to see how you structure a pipeline where new operations can be added or toggled.
3. **Batch Performance**: In production, we process hundreds of images. Your implementation should demonstrate how you could optimize for speed when handling a batch of masks simultaneously. No need to validate or benchmark performance metrics.
4. **Robustness**: Provide unit test(s) for your function/pipeline using pytest that cover edge cases (e.g., if an entirely empty mask is supplied)

What You'll Deliver

- **Source Code**: Clean, documented, and runnable Python code.
- **Unit Tests**: Provide tests that prove your function works as expected, especially for edge cases like empty masks or disconnected objects.
- **Short Analysis**: A brief explanation (README) of your architectural choices, how you would scale this for 10x more data, and how you would add more functionality i.e. post-processing steps.

The Data

You will be provided with 4 `.npz` files containing predicted masks for various car parts.

Note on Data Quality: Not every mask provided is "broken." Some predictions may already be high quality, while others will clearly exhibit the noise, holes, or artifacts mentioned above. Your pipeline should be robust enough to process all of them—improving the problematic ones without degrading those that are already correct. Use your judgment to identify which masks require intervention and which do not.

Data Structure: Each `.npz` file is a multi-channel archive. Each channel represents a single part instance. Within that channel, the pixels are binary (0 for background, or a specific `class_id` for the part). Since some parts might overlap or be predicted in separate channels, we usually "collapse" these into a single 2D semantic map for visualization.

Reference Code: You can use the *visualize.py* as a snippet to load, collapse, and visualize the data.

Timeline: We expect this task to take roughly **3-4 hours**. We are interested in your ability to prioritize clean architecture and algorithmic robustness over finding a "perfect" tuning for every single problem - remember that the minimum deliverable is a single refinement function which is robust and well tested - and does the job!